Project Report

AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)
FACULTY OF SCIENCE & TECHNOLOGY
SPRING 2025 - 2026

Computer Graphics

Section: [F]
Group No: 02

Project Report on:

SECTOR 7: PLANETARY DEFENSE - A 2D OpenGL Space Shooter

Submitted to:

Kazi Asif Ahmed
Lecturer, Department of Computer Science
American International University-Bangladesh

Submitted By:

Name	ID	CONTRIBUTION (%)
Md. Hasibul Hassan	22-46289-1	20%
J.m.efata Nusrat Borsha	23-54716-3	20%
Md. Tahmid Abtahee	23-53060-3	20%
Ashfaq Nur	23-51714-2	20%
M.Sakib Sadman Arian	23-54986-3	20%

Date of Submission: 19 - 05 - 2026

Table of content

1. Introduction

"Sector 7: Planetary Defense" is a 2D space shooter game built completely from scratch using C++ and OpenGL. The most important thing about this project is that we did **not** use any game engine like Unity or Unreal, and we did **not** use any pre-made images or textures. Every single thing you see in the game — the spaceships, the planets, the explosions, the backgrounds — is drawn using basic math and OpenGL drawing commands.

Think of it like this: instead of using a photograph of a spaceship, we told the computer "draw a triangle here, a rectangle there, color it this way" and that became our spaceship.

The game has four levels. You start at Earth, then travel to Venus, then Mercury, and finally fight a giant Sun Boss. As you move through levels, your spaceship also visually upgrades, which is a nice reward for the player.

The whole point of this project was to learn how real graphics engines work under the hood — how shapes are drawn, how things move, how collisions are detected, and how a game loop keeps everything running smoothly.

2. Objective

The main goal of this project was to take what we learned in Computer Graphics class and actually use it to build something real and playable. Here are the specific things we wanted to practice:

- **2D Transformations:** Moving, rotating, and scaling objects using matrix math commands like `glPushMatrix` and `glPopMatrix`. For example, spinning meteors use rotation transformations.
- **Drawing with Primitives:** Building complex shapes like spaceships and planets using only basic OpenGL shapes — triangles (`GL_TRIANGLES`), rectangles (`GL_QUADS`), polygons (`GL_POLYGON`), and points (`GL_POINTS`).
- **Game Loop:** Creating a smooth animation system that updates and redraws the screen about 60 times every second using a timer function.
- **State Machine:** Building a system that knows whether the game is on the menu, actively being played, showing a game over screen, or showing a victory screen — and draws the correct thing for each situation.
- **Physics and Collision:** Writing our own simple physics code to move objects and detect when bullets hit enemies.

3. Tools and Technologies Used

Tool / Technology	Purpose
C++	Main programming language used to write all game logic
OpenGL	Graphics library used to draw everything on screen
GLUT (GL Utility Toolkit)	Handles the window, keyboard input, and the timer loop
`<vector>`	Used to store lists of enemies, bullets, and particles
`<cmath>`	Used for math functions like `sinf`, `cosf`, and `sqrtf`
`<cstdlib>` and `<ctime>`	Used for random number generation (enemy spawn positions, etc.)
Visual Studio / Code::Blocks	The IDE (code editor) used to write and compile the code

There are no external game engines, no image files, and no audio libraries. Everything is pure code.

4. System Description

The project is divided into several clear components that each handle one job. Here is a breakdown of all of them:

4.1. Component 1: Game State Manager

The state manager is like the brain of the game. It keeps track of what "phase" the game is currently in. We used something called an **enum** (a list of named states) to define these phases:

```
MENU → PLAYING → LEVEL_COMPLETE → (next level) → ... → VICTORY
                                                          ↓
                                                      GAME_OVER
```

The five states are:

- **MENU** — The title screen shown at the start
- **PLAYING** — Active gameplay where the player shoots enemies
- **LEVEL_COMPLETE** — A brief pause screen shown between levels
- **GAME_OVER** — Shown when the player's health or colony health reaches zero
- **VICTORY** — Shown when the final Sun Boss is defeated

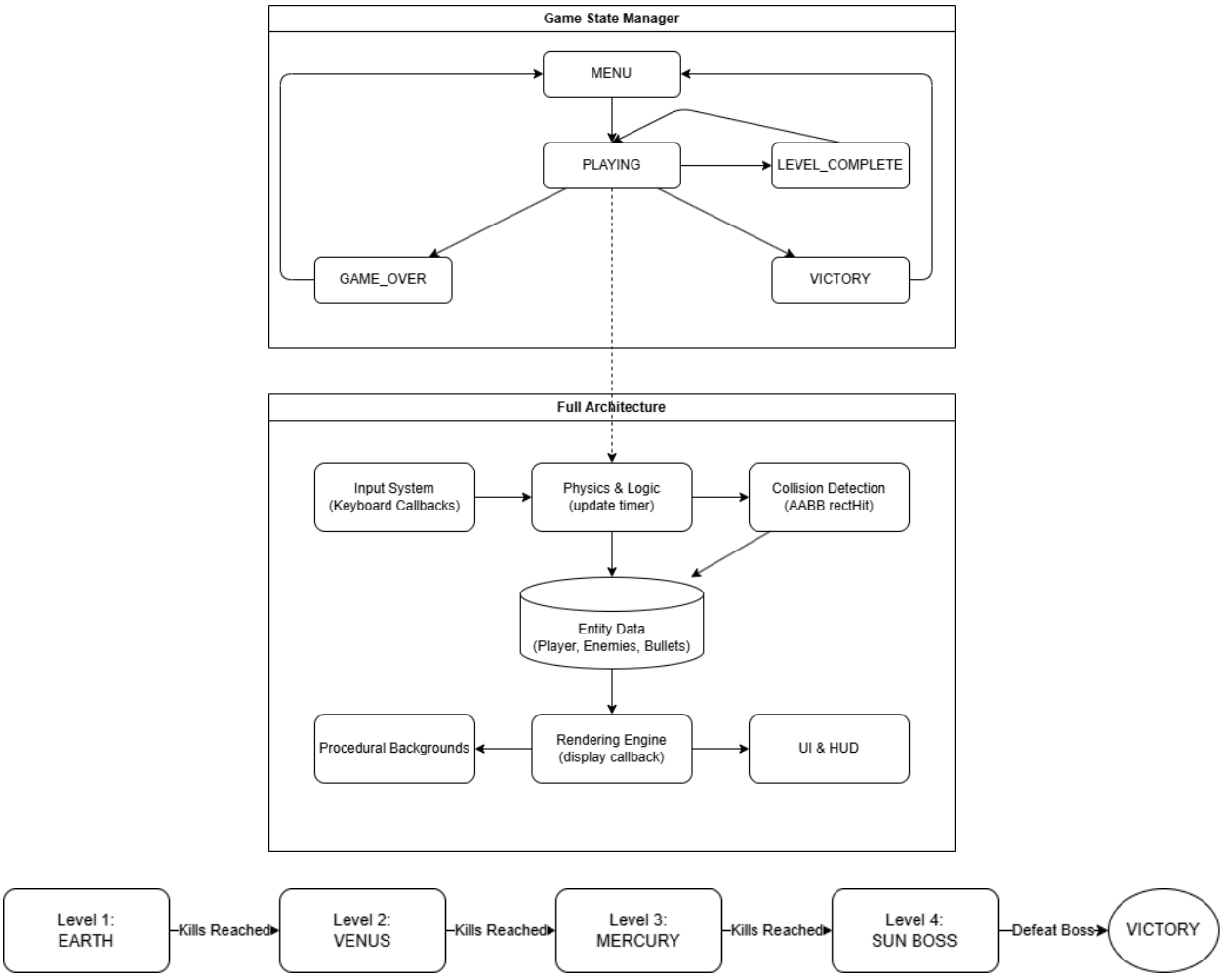

Fig: States

4.2. Component 2: Procedural Backgrounds

Earth Background (`drawEarthBackground`): This shows a blue sky with moving clouds, scrolling buildings in the distance, and trees in the foreground.

Fig: Earth Background

Venus Background (`drawVenusBackground`): This shows a dark purple and magenta atmospheric world. It has:

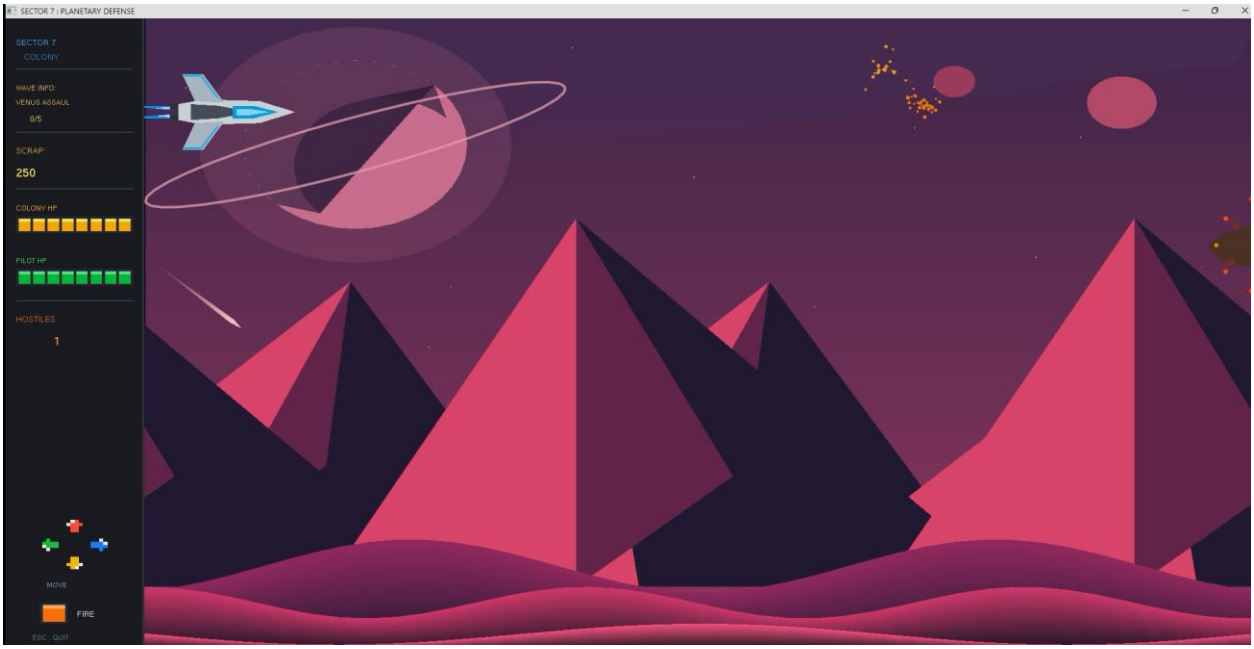

Fig: Venus Background

Mercury Background (`drawMercuryBackground`): This shows a deep space environment with The meteorites (`drawRockyMeteor`) use `glRotatef` to spin as they fly across the screen.

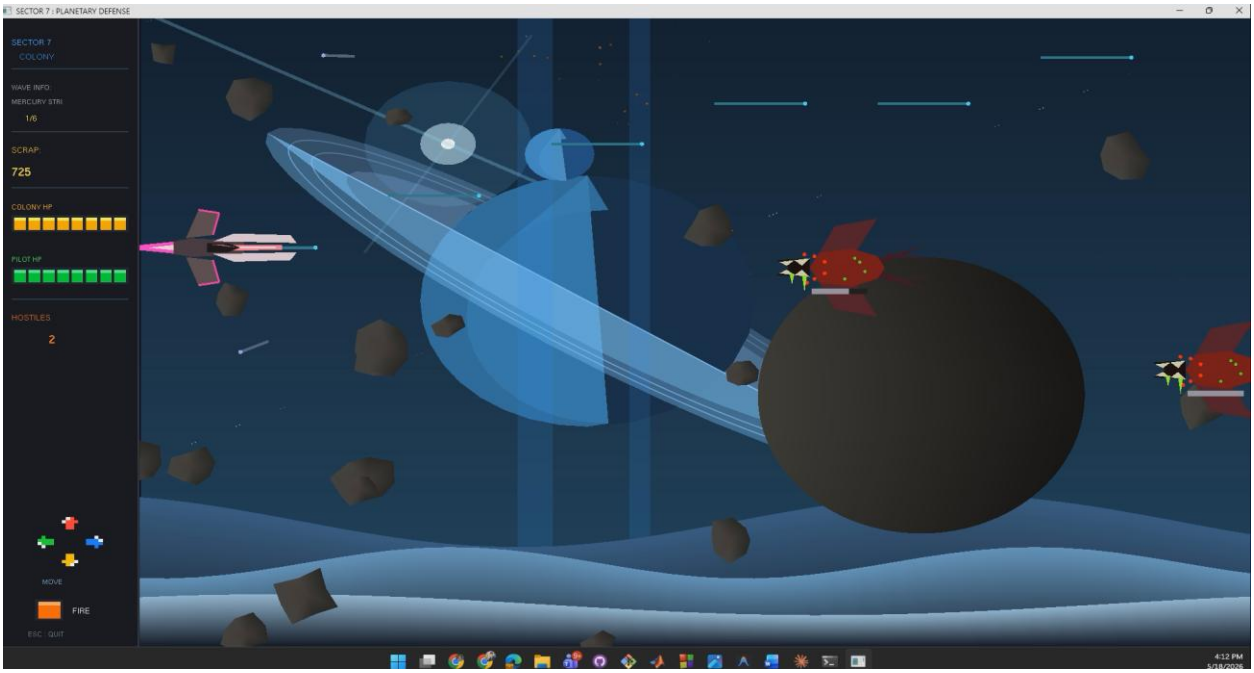

Fig: Mercury Background

Sun Boss Background (`drawSunBackground`): This shows deep black space with a massive glowing sun on the right side of the screen.

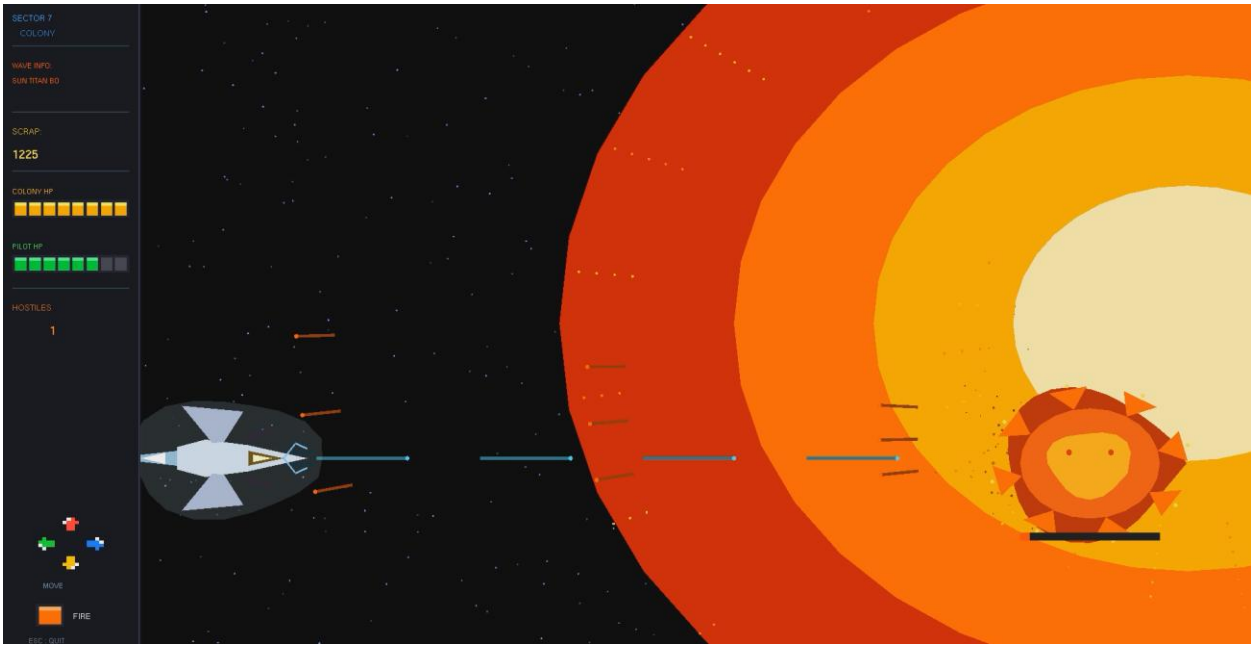

Fig: Sun Background

4.3. Component 3: Player and Enemy Entities

All game objects (player, enemies, boss) are stored in a data structure called `Entity`:

```
struct Entity {
    float x, y;       // Position
    float vx, vy;     // Velocity
    float w, h;       // Width and height (for collision)
    bool  active;     // Is this object alive?
    int   health;     // How many hits to kill
    int   type;       // What kind of enemy (1=Earth, 2=Venus, 3=Mercury,
4=Boss)
};
```

Player Ships — The player's ship changes visually with each level:

- **Earth:** `drawPlayer()` — A dark angular ship with red armor spikes and a fire aura engine
- **Venus:** `drawModernPlayer()` — A sleek white aerospace fighter with neon cyan wing edges and blue plasma exhaust
- **Mercury:** `drawMercuryPlayer()` — A neon pink interceptor with twin railgun prongs and a pulsing energy core
- **Sun Boss Level:** `drawHauntedSolarDestroyer()` — A ghostly platinum ship with spectral aura and glowing golden eyes

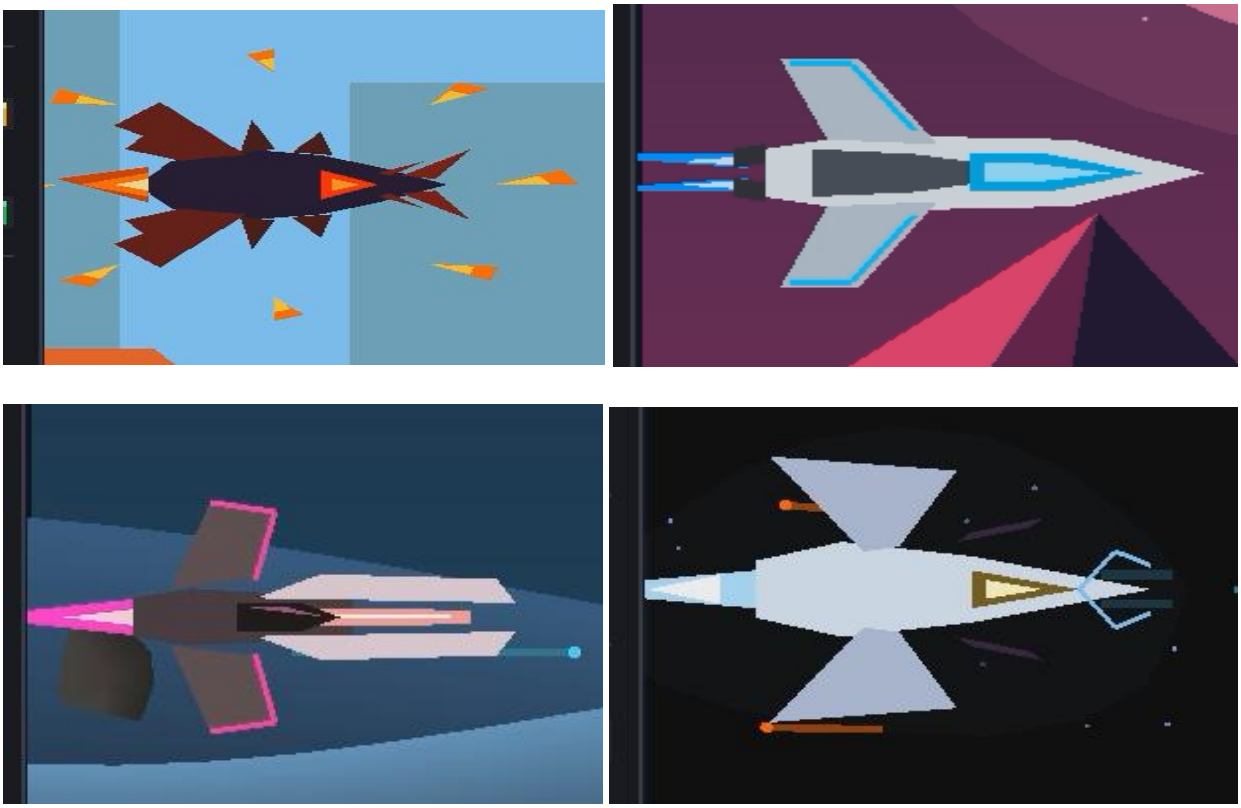

Fig: Player Ships

Enemy Ships:

- **Earth Enemy (`drawEarthEnemy`)** — A forward-swept interceptor drone in gunmetal grey with crimson wings, a glowing red sensor eye, and hostile orange exhaust
- **Venus UFO (`drawVenusUFO`)** — A large saucer-shaped ship with armor spikes, a dome, tentacles, blinking war lights, and a bottom cannon
- **Mercury Enemy (`drawMercuryUFO`)** — A horrifying biomechanical monster with bat wings, a gaping mouth with teeth, spider-like red eyes, and acid saliva drips
- **Sun Boss (`drawSunBoss`)** — A giant pulsing fire ball with rotating spike arms, glowing eyes, and its own health bar displayed beneath it

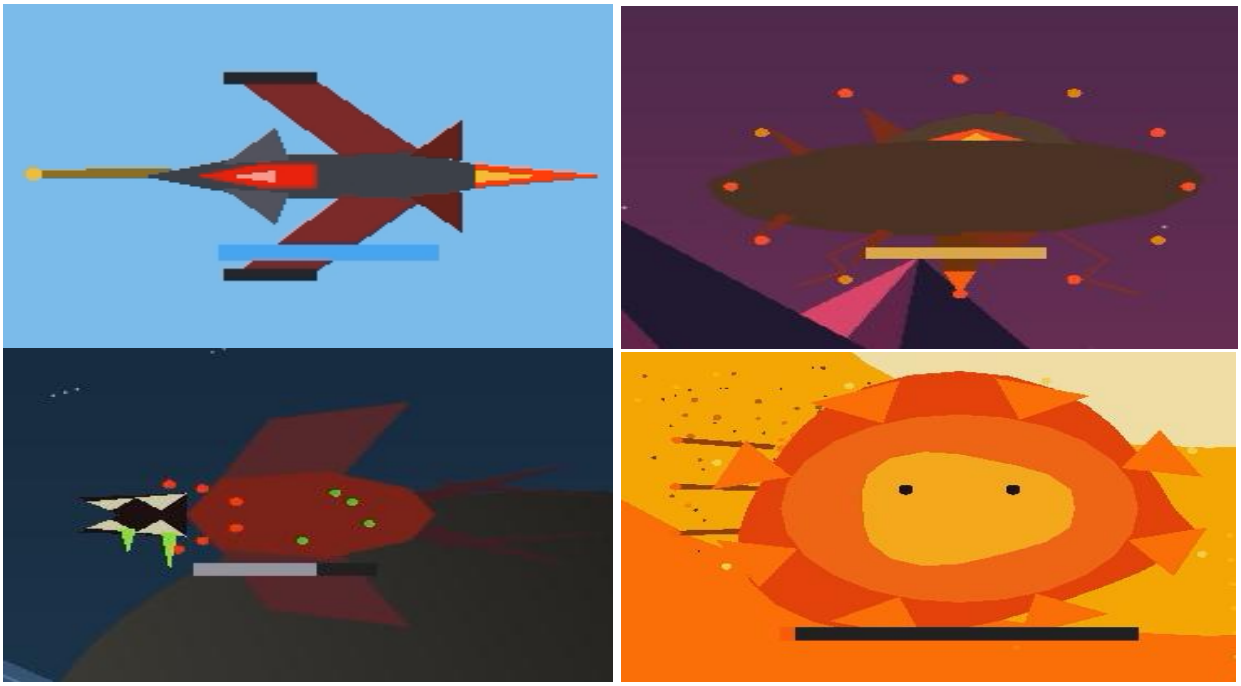

Fig: Enemy Ships

Every ship is drawn purely with `GL_POLYGON`, `GL_TRIANGLES`, `GL_QUADS`, and `GL_POINTS` — no textures at all.

4.4. Component 4: Physics and Collision System

Movement Physics: Every frame, velocities are added to positions:

```
entity.x += entity.vx
entity.y += entity.vy
```

Enemies move left at a constant speed. The player moves based on keyboard input. Bullets move in the direction they were fired. The boss bounces up and down by reversing its vertical velocity (`vy = -vy`) when it reaches the screen edges.

Bullet System: When the player presses Space, a `Bullet` struct is created and added to a list. Each bullet has its own position, velocity, and color. Every frame, all bullets move forward. A `bulletTimer` prevents the player from firing too fast — they can only shoot once every 0.13 seconds.

Enemies also fire bullets. Earth drones fire missiles every 1.5 seconds. Mercury UFOs fire plasma every 1.2 seconds. The Sun Boss fires a 3-shot spread pattern every 0.6 seconds, with each bullet aimed at the player's current position.

Fig: Player Ships Fire

Collision Detection (AABB): We use Axis-Aligned Bounding Box (AABB) collision. This is a simple rectangle-vs-rectangle check:

```
bool rectHit(float ax, float ay, float aw, float ah,
             float bx, float by, float bw, float bh) {
    return ax < bx+bw && ax+aw > bx && ay < by+bh && ay+ah > by;
}
```

In plain English: two rectangles are colliding if they overlap on both the X axis and the Y axis at the same time. This is checked for:

- Player bullets hitting enemies
- Player bullets hitting the boss
- Enemy bullets hitting the player
- Enemies physically ramming the player

Fig: Enemy Ships Attacked

When an enemy reaches the left side of the screen (past the UI panel), it damages the colony instead of being destroyed by the player.

4.5. Component 5: Particle System

The particle system creates explosion and hit effects. When something is destroyed or hit, we call `spawnParticles()` which creates many small colored dots flying outward in random directions:

```
struct Particle {
    float x, y, vx, vy;
    float life, maxLife;
    float r, g, b, size;
};
```

Each particle:

- Starts at the explosion point
- Gets a random direction (using `cosf` and `sinf` on a random angle)
- Gets a random speed between 0.5 and 4.5
- Fades out over time (its color gets multiplied by `life / maxLife`)
- Slows down each frame (velocity is multiplied by 0.94)
- Dies when `life <= 0`

Fig: Explosion and Hit effects

Different events produce different colored particles — orange-red for enemy explosions, blue for player hits, and a huge orange burst for the boss death.

4.6. Component 6: HUD and UI Panel

Side Panel (`drawUI`): A 100-pixel wide panel on the left side shows all game information:

- The game title "SECTOR 7 COLONY"
- Current level name and wave progress (e.g., "3/5 kills")
- Score (called "SCRAP" for theme)
- Colony HP shown as a segmented bar (8 orange segments)
- Pilot HP shown as a segmented bar (8 green segments)
- Number of active hostile enemies
- A pixel-art D-pad controller graphic for controls
- A retro orange fire button graphic for the shoot control

The segmented bars are a custom design — they look like old retro game health bars with gaps between each segment and a bright highlight on top.

Fig: HUD and UI Panel

Warning Flash (`drawHUD`): When colony health drops to 30% or below, a red flashing border appears around the entire game area and a "!! COLONY CRITICAL !!" warning message flashes at the top of the screen. This is done using `sinf(warnFlash)` — when the sine value is positive, the warning is visible; when it's negative, it's hidden. This creates a natural blinking rhythm.

5. Implementation Details

5.1 Initialization

Everything starts in the `main()` function. Here is exactly what happens in order:

1. `glutInit()` is called to set up the GLUT library
2. `GLUT_DOUBLE | GLUT_RGB` display mode is set — the `GLUT_DOUBLE` part means we use two framebuffers (one being drawn, one being shown) to prevent screen flickering
3. Window size is set to 900×600 pixels
4. `GL_BLEND` is enabled globally so transparency works everywhere
5. `GL_POINT_SMOOTH` and `GL_LINE_SMOOTH` are enabled so points and lines have smooth anti-aliased edges
6. `initGame()` is called which:
    - Seeds the random number generator with the current time
    - Sets the player starting position to (160, 300)

- Clears all enemy, bullet, and particle lists
- Resets score, colony HP, and pilot HP all to their starting values
- Sets the current level to EARTH
- Calls `initScenery()` to generate background objects (meteors, stars, space rocks)

5.2 Display Function

The `display()` function is called by GLUT every time the screen needs to be redrawn. Here is the exact order of operations:

1. **Clear the screen** — `glClear(GL_COLOR_BUFFER_BIT)` wipes the previous frame
2. **Check game state** and branch:
   - If `MENU` → draw the vector illustration menu background, then draw text on top of it, then return
   - If `LEVEL_COMPLETE` → draw the transition message, then return
   - If `VICTORY` → draw the victory screen, then return
   - If `GAME_OVER` → draw the game over screen, then return
   - If `PLAYING` → continue with the steps below
3. **Draw background** — `drawLevelBackground()` picks the correct background based on `currentLevel`
4. **Draw particles** — explosion effects are drawn early so ships appear on top of them
5. **Draw bullets** — each active bullet is drawn as a colored point with a short line trail
6. **Draw enemies** — each active enemy is drawn using its type-specific function
7. **Draw boss** — if the boss is active, `drawSunBoss()` is called
8. **Draw player** — the correct player ship is drawn based on the current level
9. **Draw UI** — `drawUI()` and `drawHUD()` are drawn last so they always appear on top of everything else
10. **Swap buffers** — `glutSwapBuffers()` shows the completed frame to the screen

9. Conclusion

This project proved that you do not need fancy tools to make a working game. Starting from a blank screen and building everything — the art, the physics, the game logic — using only basic math and drawing commands was a challenging but very educational experience.

We learned that things we take for granted in games (like a health bar, a scrolling background, or a flashing warning) are each their own programming problem that needs to be solved carefully. We had to think about how to make circles without a circle command, how to scroll a background without it jumping, how to make sure bullets hit enemies correctly, and how to keep the game running at a stable speed.

The most valuable lesson was understanding how a **game loop** works — the constant cycle of reading input, updating positions, checking collisions, and drawing the frame. This is the foundation of every game and graphics application, and now we understand it from first principles rather than just using an engine that hides it from us.

The project successfully demonstrates all the core concepts from the Computer Graphics course: transformations, primitives, color interpolation, matrix stack operations, animation, and real-time interactivity.

10. Limitation & Future Improvements

Current Limitations:

No Textures: Every visual is made from colored polygons. While this was intentional and educational, it means we cannot achieve the same visual detail as real games. Things like realistic metal surfaces, star nebulas, or detailed character faces are impossible with only solid color primitives.

No Audio: The game is completely silent. There are no sound effects for shooting, explosions, or level transitions, and no background music. This significantly reduces the immersive quality of the game experience.

Fixed Window Size: The game is hard-coded to 900×600 pixels. If the window is resized, the game logic does not scale — objects would appear in the wrong proportions and the HUD would break.

Simple AI: Enemies only move in a straight line and fire at fixed time intervals. They do not dodge player bullets, form attack formations, or change behavior based on the situation.

No Save System: There is no way to save progress. Every time the game is closed, the player starts over from Level 1.

Future Improvements:

- **Texture Mapping:** Using a library like SOIL or stb_image to load `.png` image files and map them onto the ship polygons would dramatically improve visual quality.
- **Audio System:** Adding OpenAL or the irrKlang library would allow laser sounds, explosion booms, and atmospheric background music for each level.
- **Resolution Scaling:** Replacing all hard-coded pixel values with values calculated relative to the window size, so the game looks correct at any resolution.
- **Smarter Enemy AI:** Enemies could be given formation patterns, evasion behaviors, or different attack phases to make gameplay more interesting and challenging.
- **Power-Ups:** Items that drop from defeated enemies could give temporary abilities like a shield, rapid fire, or a spread shot.
- **High Score System:** Using file I/O (`fstream`) to save the top scores to a text file, so high scores persist between sessions.
- **More Levels and Enemy Types:** The modular architecture of the code makes it easy to add new background functions and new enemy drawing functions for additional planets.

References

[1] F. S. Hill and S. M. Kelley, *Computer Graphics Using OpenGL*, 3rd ed. Pearson Prentice Hall, 2006.

[2] OpenAI / Gemini AI Assistance for Code Documentation and Report Structuring.

[3] "OpenGL API Documentation," Khronos Group. Available: https://www.opengl.org/documentation/